

Modellierung des DRAUSP-Problems als Reinforcement-Learning-Umgebung Gymnasium-Umgebung mit DQN und Action Masking

Bachelorarbeit

May 8, 2026

- 1 Das DRAUSP-Problem
- 2 Grundlagen: Reinforcement Learning
- 3 Modellierung als RL-Problem
- 4 Das Problem der ungünstigen Aktionen
- 5 Gymnasium-Implementierung
- 6 Zusammenfassung & Ausblick

Dynamic Resource Allocation Under Stochastic Prices (DRAUSP)

Problemstellung

Gegeben T_d Anfragen, die sequenziell eintreffen. Jede Anfrage i besitzt:

- einen **Erloes** $r_i \in \mathbb{R}$
- einen **Kapazitaetsbedarf** $q_i \in \mathbb{N}^K$ ueber K Zeitslots

Entscheidung

Fuer jede Anfrage muss **sofort und unwiderruflich** entschieden werden:

- **Ablehnen** $\rightarrow$ Erloes = 0
- **Akzeptieren ab Slot k** $\rightarrow$ Erloes = r_i , Kapazitaeten werden reduziert

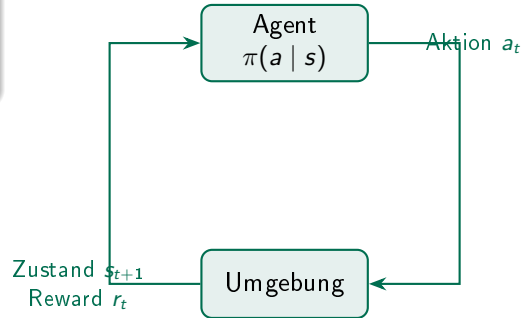
Ziel: Maximierung des **Gesamterloesei** unter Einhaltung der Kapazitaetsschranken C_k .

Kernidee

Ein **Agent** interagiert mit einer **Umgebung** und lernt durch Versuch und Irrtum eine optimale **Policy** π .

Zentrale Komponenten:

- Zustand $s \in \mathcal{S}$
- Aktion $a \in \mathcal{A}$
- Reward r
- Policy $\pi(a | s)$
- Q-Funktion $Q(s, a)$



Q-Learning – Policy aus Q-Werten

Q-Funktion

$Q(s, a)$ bewertet, wie gut es ist, in Zustand s die Aktion a zu waehlen – unter Beruecksichtigung **aller zukuenftigen Rewards**.

Optimale Policy:

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}} Q^*(s, a)$$

Der Agent waehlt stets die Aktion mit dem **hoechsten Q-Wert**.

Lernregel (Bellman):

$$Q(s, a) \leftarrow r + \gamma \cdot \max_{a'} Q(s', a')$$

Q-Werte in Zustand s

$$Q(s, a_0) = 12$$

$$Q(s, a_1) = 47 \leftarrow \pi^*$$

$$Q(s, a_2) = 31$$

Deep Q-Network (DQN)

Warum ein neuronales Netz?

Der Zustandsraum ist zu gross, um $Q(s, a)$ tabellarisch zu speichern → ein **neuronales Netz** approximiert $Q(s, a)$ fuer alle Aktionen gleichzeitig.

Schluesselideen des DQN:

- **Replay Buffer:** Erfahrungen (s, a, r, s') werden gespeichert und zufaellig gesampelt → bricht zeitliche Korrelationen auf
- **Target-Netz:** zweites, seltener aktualisiertes Netz stabilisiert das Training
- **ϵ -Greedy:** Balance zwischen Exploration und Exploitation
- **Gradient Clipping:** verhindert explodierende Gradienten

Soft Update des Target-Netzes

$$\theta_{\text{target}} \leftarrow \tau \cdot \theta_{\text{policy}} + (1 - \tau) \cdot \theta_{\text{target}}, \quad \tau \ll 1$$

Zustandsvektor $s \in \mathcal{S} \subseteq \mathbb{R}^{2K+2}$

$$s = (t, \hat{C}_1, \dots, \hat{C}_K, r, q_1, \dots, q_K)$$

Komponente	Bedeutung	Wertebereich
t	aktueller Zeitschritt	$\{1, \dots, T_d\}$
$\hat{C}_k$	verbleibende Kapazitaet in Slot k	$\{0, \dots, C_k\}$
r	Erloes der aktuellen Anfrage	$\mathbb{R}_{\geq 0}$
q_k	benoetigter Bedarf in Slot k	$\{0, \dots, C_k\}$

Initialzustand:

$$s_1 = (1, C_1, \dots, C_K, r_1, q_1^1, \dots, q_1^K)$$

Aktionsraum und Uebergangsfunktion

Aktionsraum $\mathcal{A} = \{0, 1, \dots, K\}$

- **Aktion 0:** Anfrage ablehnen $\rightarrow$ Kapazitaeten unveraendert
- **Aktion $k > 0$:** Anfrage ab Slot k einplanen $\rightarrow \hat{C}_{k+i} \leftarrow \hat{C}_{k+i} - q_i$ fuer $i = 0, \dots, \ell_q - 1$

Reward-Funktion

$$R(s, a) = \begin{cases} r & \text{Akzeptieren, Kapazitaet ausreichend} \\ 0 & \text{Ablehnen} \\ -100 & \text{unguelte Aktion (Kapazitaetsverletzung)} \end{cases}$$

Episode endet (terminated = True), wenn:

- alle T_d Anfragen abgearbeitet wurden, **oder**
- eine ungueltige Aktion gewaehlt wurde

Das Problem der ungueltigen Aktionen

Problem

Nicht jede Aktion $a \in \mathcal{A}$ ist in jedem Zustand **zulaessig**!

Zwei Arten von ungueltigen Aktionen:

- 1 **Slot zu weit rechts:** $\ell_q + a - 1 > K$
Die Anfrage wurde ueber den letzten Slot hinausragen.
- 2 **Kapazitaetsverletzung:** $\exists i : \hat{C}_{a+i} < q_i$
Die verbleibende Kapazitaet reicht nicht aus.

Konsequenz fuer Standard-DQN

Ein Standard-DQN ohne Behandlung ungueltiger Aktionen waehlt per $\arg \max$ ggf. eine unzulaessige Aktion $\rightarrow$ Reward = -100 $\rightarrow$ **instabiles, langsames Training**.

Idee

Ungueltige Aktionen erhalten **vor der Auswahl** einen Q-Wert von $-\infty$, sodass $\arg \max$ sie niemals waehlt.

Action Masking muss an *zwei* Stellen erfolgen:

1. Aktionsauswahl

$$\tilde{Q}(s, a) = \begin{cases} Q_{\pi}(s, a) & a \in \mathcal{A}_{\text{valid}}(s) \\ -\infty & \text{sonst} \end{cases}$$

$$a^* = \arg \max_a \tilde{Q}(s, a)$$

2. Bellman-Target (Training)

$$\tilde{Q}_{\text{target}}(s', a') = \begin{cases} Q_{\text{target}}(s', a') & a' \in \mathcal{A}_{\text{valid}}(s') \\ -\infty & \text{sonst} \end{cases}$$

$$y = r + \gamma \cdot \max_{a'} \tilde{Q}_{\text{target}}(s', a')$$

Gymnasium

Standardisiertes Python-Interface fuer RL-Umgebungen – kompatibel mit gaengigen RL-Bibliotheken.

Pflichtmethoden (Gymnasium-Standard):

Methode	Rueckgabe	Bedeutung
<code>reset()</code>	<code>obs, info</code>	Neue Episode starten
<code>step(action)</code>	<code>obs, r, terminated, truncated, info</code>	Schritt ausfuehren
<code>render()</code>	–	Zustand visualisieren

Eigene Erweiterung fuer Action Masking:

`get_valid_actions()` → `list[int]` Gibt alle zulaessigen Aktionen im aktuellen Zustand zurueck.

Beitrag

Modellierung des DRAUSP-Problems als Reinforcement-Learning-Problem mit einer standardisierten Gymnasium-Umgebung.

Geplanter Vergleich:

Methode	Typ	Action Masking
Random Policy	Baseline	–
DQN ohne Masking	RL	nein
DQN mit Masking	RL	ja
Monotones DQN	RL (Erweiterung)	ja

① Allgemeiner Aktionsraum

Bisher: feste Requestgroesse ℓ_q . Geplant: variable Requestgroessen

② Monotone Neural Networks

Architektur erzwingt strukturelle Monotonie in Kapazitaeten und Erloes:

$$\hat{C}_k \uparrow \Rightarrow Q(s, a) \uparrow \qquad r \uparrow \Rightarrow Q(s, a) \uparrow$$

③ Evaluation auf Benchmark-Instanzen

- **Konvergenzgeschwindigkeit:** Anzahl Episoden bis zur Stabilisierung
- **Trainings-Stabilitaet:** Varianz des kumulierten Rewards (Sample-Varianz)
- **Sample-Effizienz:** erreichter Reward pro Umgebungsinteraktion
- **Generalisierung:** Out-of-sample Performance auf ungesehenen Instanzen

④ Hyperparameter-Analyse

Lernrate α , Diskontierungsfaktor γ , Soft-Update-Rate τ , Replay-Buffer-Groesse, Netzwerkarchitektur

Vielen Dank!

Fragen?